IUT Robert Schuman
Institut universitaire de technologie
Université de Strasbourg

P11 – Initiation à la programmation

-

Les tableaux

Hervé L'helguen

Plan

1. Introduction

Introduction

Partons d'une situation où l'on veut stocker 5 notes obtenues par un étudiant, et calculer sa moyenne.

note1	note2	note3	note4	note5
11	7	14.5	10	14

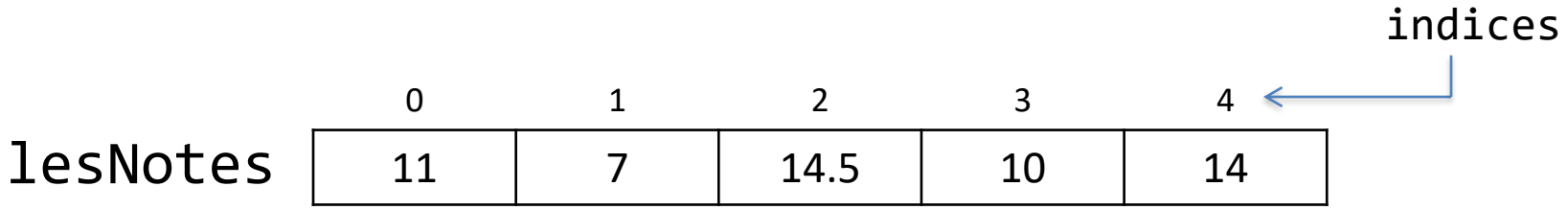
On s'aperçoit que l'on est obligé d'utiliser 5 variables indépendantes qui vont avoir le même rôle : stocker une note. Si avec 5 notes c'est encore gérable, que deviendrait l'algorithme avec 20 notes, 100 notes et plus... ?

La solution est de déclarer une seule variable composée de 5 cases : **un tableau**

`lesNotes`					
	11	7	14.5	10	14

Introduction

L'énorme avantage est de pouvoir accéder à chaque case du tableau (élément) par son indice qui représente le n° d'ordre des cases.



Ainsi `lesNotes[0]` contient 11

`lesNotes[3]` contient 10

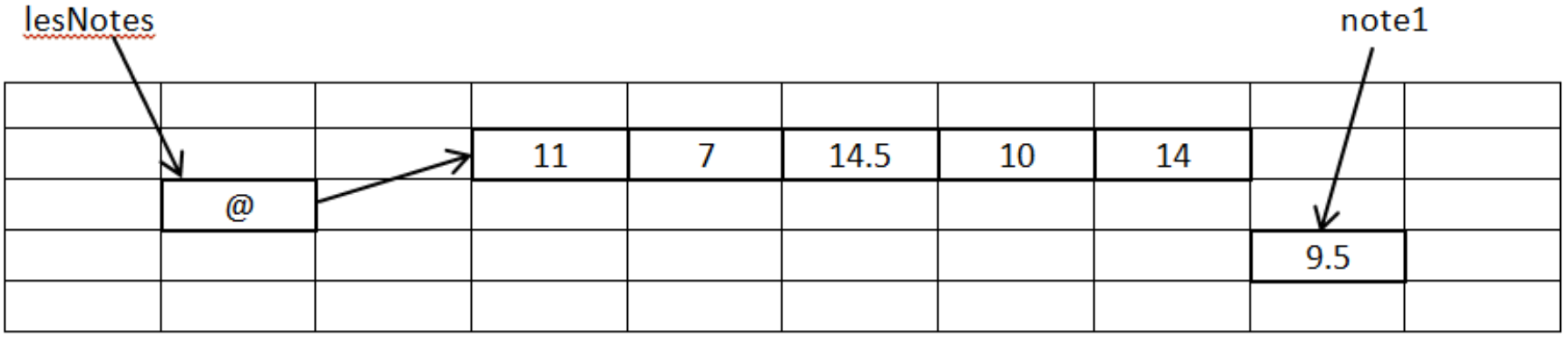
Remarque : les indices sont numérotés à partir de 0

Plan

1. Introduction
2. Comprendre ce qu'est un tableau

Comprendre ce qu'est un tableau

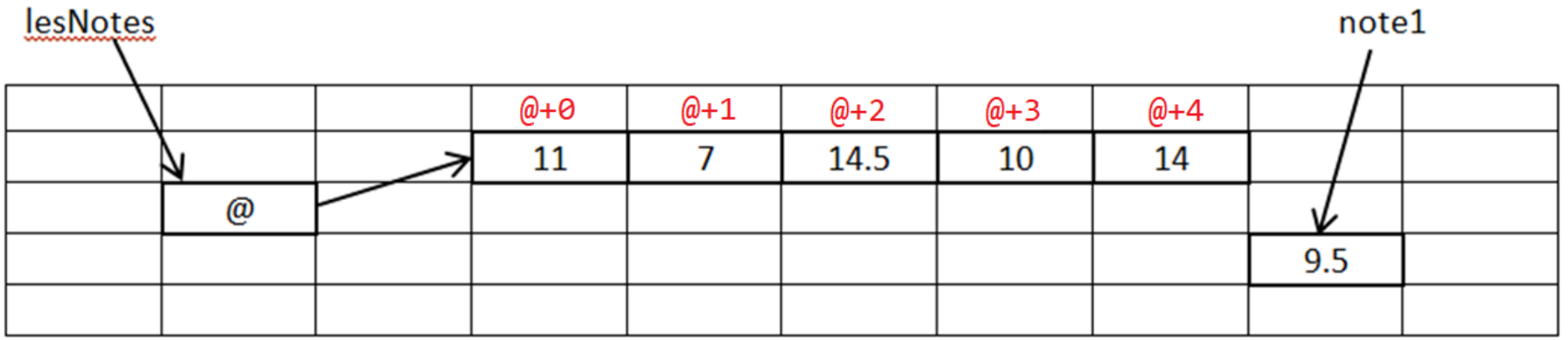
Schématiquement on peut représenter le tableau `lesNotes` comme ceci :



La variable lesNotes va contenir l'adresse de la 1ère case du tableau, et les cases suivantes vont se suivre (elles sont contigües).

Comprendre ce qu'est un tableau

Schématiquement on peut représenter le tableau `lesNotes` comme ceci :



La variable lesNotes va contenir l'adresse de la 1ère case du tableau, et les cases suivantes vont se suivre (elles sont contigües).

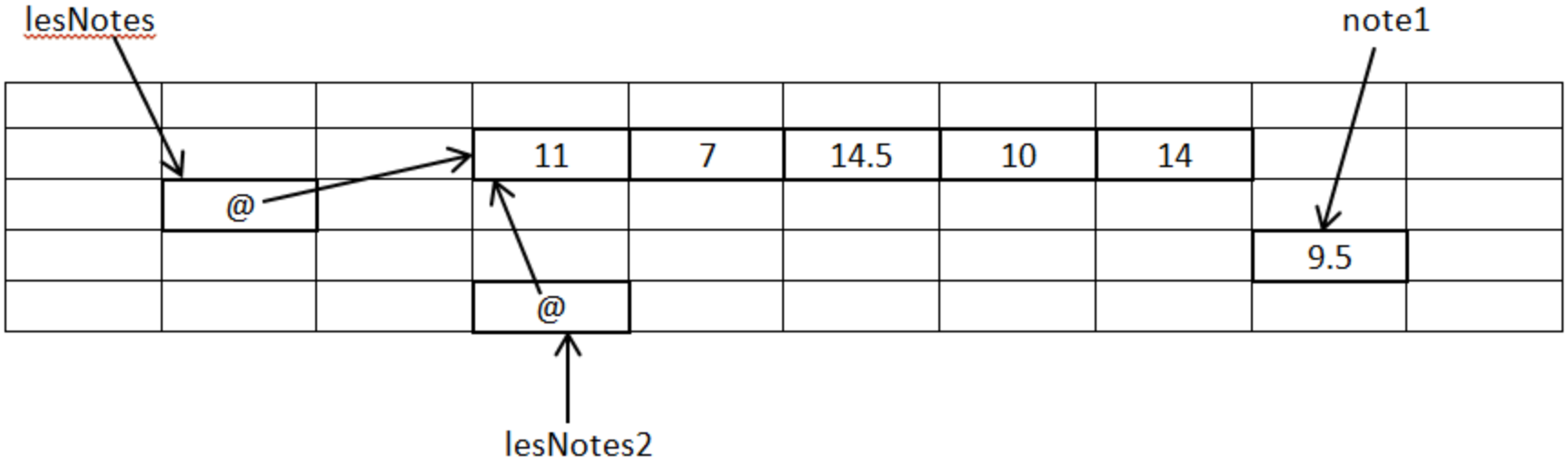
Ne pas confondre : lesNotes qui contient une adresse

lesNotes[2] qui contient une valeur (ici 14.5)

Comprendre ce qu'est un tableau

Attention : si je fais `lesNotes2 = lesNotes`

Voici ce qui se passe :



C'est l'adresse contenue dans `lesNotes` qui est recopiée dans `lesNotes2` qui pointe désormais vers les mêmes cases.

Ceci aura son importance pour les fonctions et les procédures.

Plan

1. Introduction
2. Comprendre ce qu'est un tableau
3. La déclaration d'un tableau

La déclaration d'un tableau

Je veux déclarer le tableau lesNotes dans un algorithme :

```
Algorithme testTableau
    Déclarations
        Variables
            lesNotes : tableau de 5 réels
Début
    …
Fin
```

- la taille du tableau est fixe, les cases iront de lesNotes[0] à lesNotes[4]
- les cases du tableau ne pourront stocker que des réels

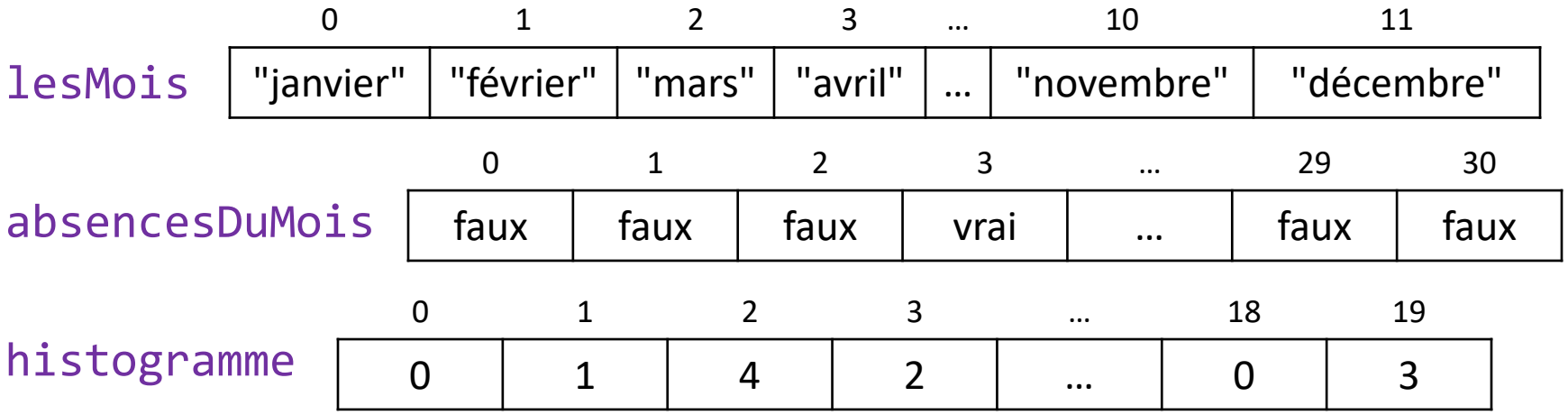
La déclaration d'un tableau

Je veux déclarer le tableau lesNotes et l'initialiser directement lors de la déclaration :

```
Algorithme testTableau
    Déclarations
        Variables
            lesNotes : tableau de 5 réels initialisé
                        à {11,7,14.5,10,14}
Début
    …
Fin
```

La déclaration d'un tableau

Voici 3 tableaux :



Quelle serait leur déclaration dans un algorithme ?

La déclaration d'un tableau

```
Algorithme testTableau
    Déclarations
        Variables
            lesMois : tableau de 12 chaînes
            absentJour : tableau de 31 booléens
            histogramme : tableau de 20 entiers
```

lesMois

0	1	2	3	…	10	11
"janvier"	"février"	"mars"	"avril"	…	"novembre"	"décembre"

absencesDuMois

0	1	2	3	…	29	30
faux	faux	faux	vrai	…	faux	faux

histogramme

0	1	2	3	…	18	19
0	1	4	2	…	0	3

Plan

1. Introduction
2. Comprendre ce qu'est un tableau
3. La déclaration d'un tableau
4. L'utilisation d'un tableau

L'utilisation d'un tableau

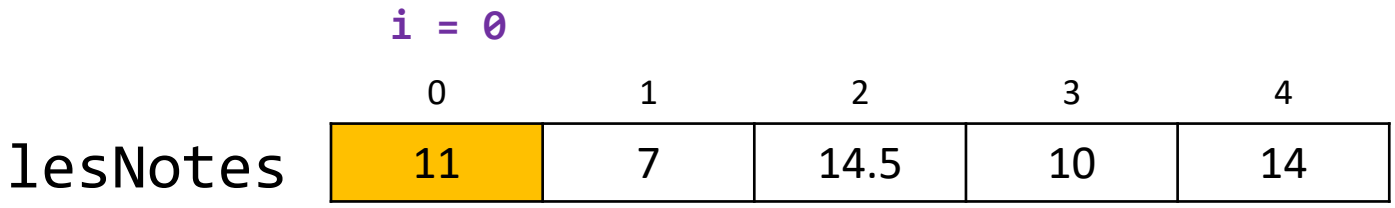
Je veux afficher le contenu du tableau lesNotes

```
Algorithme testTableau
    Déclarations
        Variables
            lesNotes : tableau de 5 réels initialisé
                        à {11,7,14.5,10,14}
            i : entier
Début
    Pour i de 0 à 4
        Afficher lesNotes[i]
    FinPour
Fin
```

5-1 pour ne pas dépasser la limite du tableau (lesNotes[5] n'existe pas)

L'utilisation d'un tableau

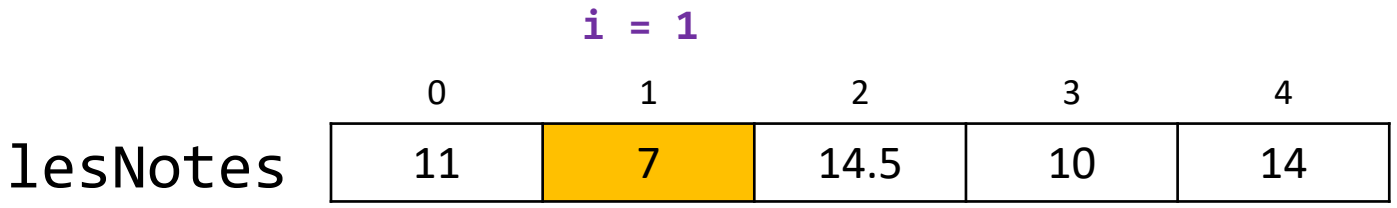
```
Pour i de 0 à 4
    Afficher lesNotes[i]
FinPour
```




```
Afficher lesNotes[0]
```

L'utilisation d'un tableau

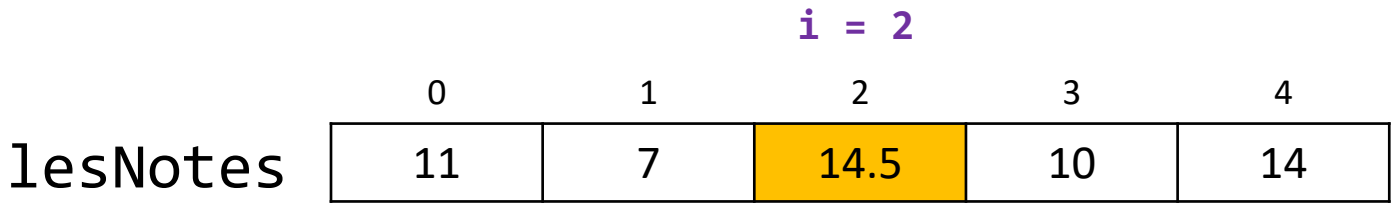
```
Pour i de 0 à 4
    Afficher lesNotes[i]
FinPour
```




```
Afficher lesNotes[1]
```

L'utilisation d'un tableau

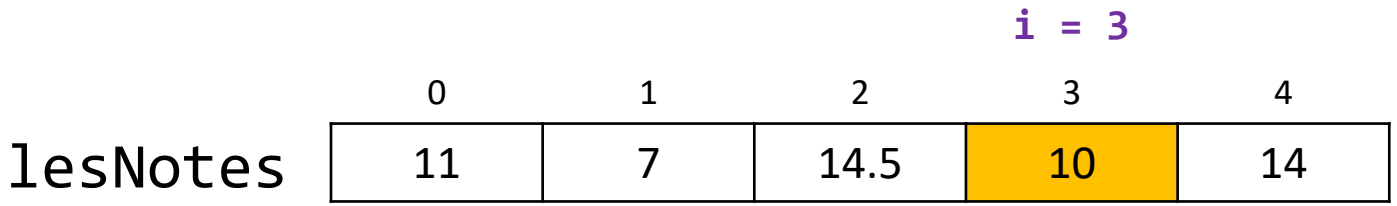
```
Pour i de 0 à 4
    Afficher lesNotes[i]
FinPour
```




```
Afficher lesNotes[2]
```

L'utilisation d'un tableau

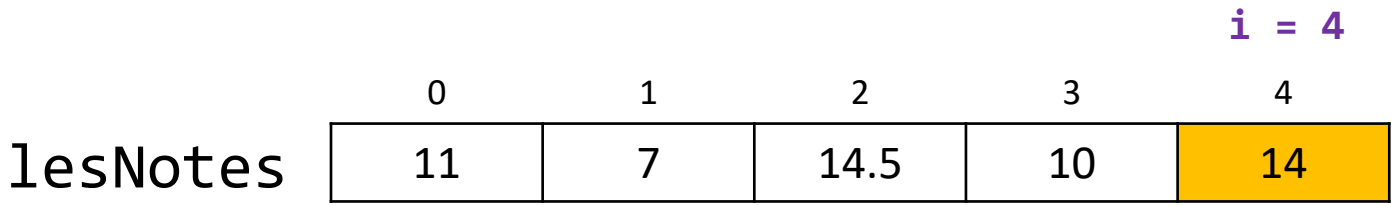
```
Pour i de 0 à 4
    Afficher lesNotes[i]
FinPour
```




```
Afficher lesNotes[3]
```

L'utilisation d'un tableau

```
Pour i de 0 à 4
    Afficher lesNotes[i]
FinPour
```




```
Afficher lesNotes[4]
```

L'utilisation d'un tableau

Je veux initialiser à 0 le contenu du tableau lesNotes

```
Algorithme testTableau
    Déclarations
        Variables
            lesNotes : tableau de 107 réels
            i : entier
Début
    Pour i de 0 à 106
        lesNotes[i] = 0     //Affectation directe dans la case
    FinPour
Fin
```

Tableau à 1 dimension en C#

```
class testTableau
{
    static void Main()
    {
        //1-Déclarations
        float[] lesNotes; //tableau de float

        //2-Création du tableau
        lesNotes = new float[5];
    }
}
```

je crée un tableau composé de 5 cases. Chaque case stockera un float

```
class testTableau
{
    static void Main()
    {
        //Déclaration et initialisation
        float[] lesNotes = {11,7,14.5f,10,14};
    }
}
```

Dans ce cas, inutile de faire le new, le tableau est créé avec le nombre de cases égal au nombre d'éléments de la liste (ici 5).

Plan

1. Introduction
2. Comprendre ce qu'est un tableau
3. La déclaration d'un tableau
4. L'utilisation d'un tableau
5. Tableau à 2 dimensions

Tableau à 2 dimensions

Repartons de notre contexte initial où nous avons créé le tableau lesNotes pour stocker les notes des étudiants. Un nouveau besoin apparait, je veux maintenant stocker les notes sur les 2 semestres :

`lesNotesS1`

0	1	2	3	4
11	7	14.5	10	14

`lesNotesS2`

0	1	2	3	4
9	8.5	11	12	14

Après avoir utilisé deux tableaux pour les deux semestres, on cherche à regrouper ces données dans une seule structure plus adaptée : **un tableau à deux dimensions.**

Tableau à 2 dimensions

Version tableau 1 dimension :

`lesNotesS1`

0	1	2	3	4
11	7	14.5	10	14

`lesNotesS2`

0	1	2	3	4
9	8.5	11	12	14

Version tableau 2 dimensions :

`lesNotes`

	0	1	2	3	4
0	11	7	14.5	10	14
1	9	8.5	11	12	14

Composé de 2 lignes (1 pour chaque semestre) et 5 colonnes

Tableau à 2 dimensions

`lesNotes`

	0	1	2	3	4
0	11	7	14.5	10	14
1	9	8.5	11	12	14

De même que pour les colonnes, les lignes sont également numérotées (à partir de 0).

`lesNotes[1,3]` représente la case se trouvant à la ligne 1 et à la colonne 3

Remarque : le 1er indice est toujours celui des lignes et le 2ème celui des colonnes.

Astuce : pensez au président Lincoln

line | column

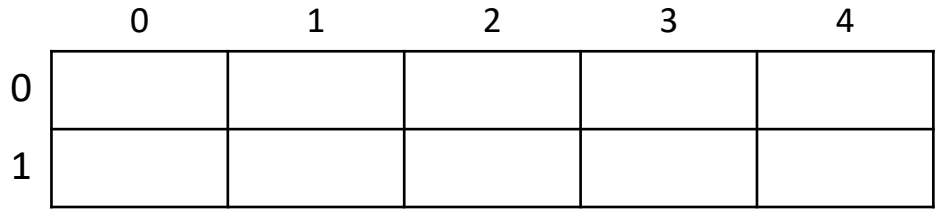
Plan

1. Introduction
2. Comprendre ce qu'est un tableau
3. La déclaration d'un tableau
4. L'utilisation d'un tableau
5. Tableau à 2 dimensions
6. La déclaration d'un tableau à 2 dimensions

La déclaration d'un tableau à 2 dimensions

Je veux déclarer le tableau lesNotes dans un algorithme :

```
Algorithme testTableau
    Déclarations
      Variables
         lesNotes : tableau de 2 lignes et 5 colonnes de réels
Début
    …
Fin
```



- la taille du tableau est fixe, les 10 cases iront de lesNotes[0,0] à lesNotes[1,4]
- les cases du tableau ne pourront stocker que des réels

La déclaration d'un tableau

Je veux déclarer le tableau lesNotes et l'initialiser directement lors de la déclaration :

```
Algorithme testTableau
    Déclarations
      Variables
        lesNotes : tableau de 2 lignes et 5 colonnes de réels
          initialisé à {{11,7,14.5,10,14},
                        {9,8.5,11,12,14}}
Début
    …
Fin
```

	0	1	2	3	4
0	11	7	14.5	10	14
1	9	8.5	11	12	14

Plan

1. Introduction
2. Comprendre ce qu'est un tableau
3. La déclaration d'un tableau
4. L'utilisation d'un tableau
5. Tableau à 2 dimensions
6. La déclaration d'un tableau à 2 dimensions
7. L'utilisation d'un tableau à 2 dimensions

L'utilisation d'un tableau à 2 dimensions

Je veux afficher le contenu du tableau lesNotes :

```
Algorithme testTableau
    Déclarations
      Variables
         lesNotes : tableau de 2 lignes et 5 colonnes de réels
            initialisé à {{11,7,14.5,10,14},{9,8.5,11,12,14}}
         i, j : entiers     //i va parcourir les indices des lignes
Début                       //j va parcourir les indices des colonnes
    //Parcours des lignes
    Pour i de 0 à 1
       //Parcours des colonnes
       Pour j de 0 à 4
          Afficher lesNotes[i,j]
       FinPour
    FinPour
Fin
```

L'utilisation d'un tableau à 2 dimensions

```
Pour i de 0 à 1
    //Parcours des colonnes
    Pour j de 0 à 4
        Afficher lesNotes[i,j]
    FinPour
FinPour
```

lesNotes

i = 0

j = 0

	0	1	2	3	4
0	11	7	14.5	10	14
1	9	8.5	11	12	14

```
Afficher lesNotes[0,0]
```

L'utilisation d'un tableau à 2 dimensions

```
Pour i de 0 à 1
    //Parcours des colonnes
    Pour j de 0 à 4
        Afficher lesNotes[i,j]
    FinPour
FinPour
```

lesNotes

i = 0

j = 1

	0	1	2	3	4
0	11	7	14.5	10	14
1	9	8.5	11	12	14

```
Afficher lesNotes[0,1]
```

L'utilisation d'un tableau à 2 dimensions

```
Pour i de 0 à 1
    //Parcours des colonnes
    Pour j de 0 à 4
        Afficher lesNotes[i,j]
    FinPour
FinPour
```

lesNotes

i = 0

j = 2

	0	1	2	3	4
0	11	7	14.5	10	14
1	9	8.5	11	12	14

```
Afficher lesNotes[0,2]
```

L'utilisation d'un tableau à 2 dimensions

```
Pour i de 0 à 1
    //Parcours des colonnes
    Pour j de 0 à 4
        Afficher lesNotes[i,j]
    FinPour
FinPour
```

lesNotes

i = 0

j = 3

	0	1	2	3	4
0	11	7	14.5	10	14
1	9	8.5	11	12	14

```
Afficher lesNotes[0,3]
```

L'utilisation d'un tableau à 2 dimensions

```
Pour i de 0 à 1
    //Parcours des colonnes
    Pour j de 0 à 4
        Afficher lesNotes[i,j]
    FinPour
FinPour
```

lesNotes

i = 0

j = 4

	0	1	2	3	4
0	11	7	14.5	10	14
1	9	8.5	11	12	14

```
Afficher lesNotes[0,4]
```

L'utilisation d'un tableau à 2 dimensions

```
Pour i de 0 à 1
    //Parcours des colonnes
    Pour j de 0 à 4
        Afficher lesNotes[i,j]
    FinPour
FinPour
```

lesNotes

j = 0

i = 1		0	1	2	3	4
	0	11	7	14.5	10	14
	1	9	8.5	11	12	14

```
Afficher lesNotes[1,0]
```

L'utilisation d'un tableau à 2 dimensions

```
Pour i de 0 à 1
    //Parcours des colonnes
    Pour j de 0 à 4
        Afficher lesNotes[i,j]
    FinPour
FinPour
```

lesNotes

j = 1

i = 1	0	1	2	3	4
0	11	7	14.5	10	14
1	9	8.5	11	12	14

```
Afficher lesNotes[1,1]
```

L'utilisation d'un tableau à 2 dimensions

```
Pour i de 0 à 1
    //Parcours des colonnes
    Pour j de 0 à 4
        Afficher lesNotes[i,j]
    FinPour
FinPour
```

lesNotes

j = 2

		0	1	2	3	4
	0	11	7	14.5	10	14
i = 1	1	9	8.5	11	12	14

```
Afficher lesNotes[1,2]
```

L'utilisation d'un tableau à 2 dimensions

```
Pour i de 0 à 1
    //Parcours des colonnes
    Pour j de 0 à 4
        Afficher lesNotes[i,j]
    FinPour
FinPour
```

lesNotes

j = 3

		0	1	2	3	4
	0	11	7	14.5	10	14
i = 1	1	9	8.5	11	12	14

```
Afficher lesNotes[1,3]
```

L'utilisation d'un tableau à 2 dimensions

```
Pour i de 0 à 1
    //Parcours des colonnes
    Pour j de 0 à 4
        Afficher lesNotes[i,j]
    FinPour
FinPour
```

lesNotes

j = 4

		0	1	2	3	4
	0	11	7	14.5	10	14
i = 1	1	9	8.5	11	12	14

```
Afficher lesNotes[1,4]
```

Tableau à 2 dimensions en C#

```
class testTableau
{
    static void Main()
    {
        //1-Déclarations
        float[,] lesNotesAnnee;     //tableau 2D de float

        //2-Création du tableau
        lesNotesAnnee = new float[3,5];
    }
}
```

je crée un tableau composé
de 3 lignes et 5 colonnes
Chaque case stockera un float

```
class testTableau
{
    static void Main()
    {
        //Déclaration et initialisation
        float[,] lesNotesAnnee = {{11,7,14.5f,10,14},
                                  {10,7.5f,15,09,14},
                                  {1,7,12.5f,11,13}};
    }
}
```

Dans ce cas, inutile de faire le new, le tableau
est créé avec le nombre de lignes et colonnes
correspondant aux valeurs d'initialisation.

Plan

1. Introduction
2. Comprendre ce qu'est un tableau
3. La déclaration d'un tableau
4. L'utilisation d'un tableau
5. Tableau à 2 dimensions
6. La déclaration d'un tableau à 2 dimensions
7. L'utilisation d'un tableau à 2 dimensions
8. Passage d'un tableau en paramètre

Passage d'un tableau en paramètre

Je veux passer le tableau lesNotes à une procédure afficherTableau()

```
//L'appel
Algorithme testTableau
    Déclarations
        Variables
            lesNotes : tableau de 5 réels initialisé
                       à {11,7,14.5,10,14}
Début
    afficherTableau(lesNotes)
Fin
```

Passage d'un tableau en paramètre

//Côté procédure

```
Procédure afficherTableau(tab : tableau de réels)
    Déclarations
        Variables
            i : entier
 Début
    Pour i de 0 à ??
        afficher tab[i] + "-"
    FinPour
 Fin
```

Ici je reçois un tableau de réels
dont je ne connais pas la taille
mais qui rend plus générique
La procédure

Passage d'un tableau en paramètre

Solution 1 : Considérer qu'il existe une fonction pour récupérer la taille (ce qui sera le cas en C#) :

```
Procédure afficherTableau(tab : tableau de réels)
   Déclarations
      Variables
         i : entier
 Début
   Pour i de 0 à taille(tab) - 1
      afficher tab[i] – "-"
   FinPour
 Fin
```

Passage d'un tableau en paramètre

Solution 2 : Passer la taille en paramètre (ce qui sera le cas en langage C)

```
Procédure afficherTableau(tab : tableau de réels, taille : entier)
    Déclarations
        Variables
            i : entier
 Début
    Pour i de 0 à taille - 1
        afficher tab[i]
    FinPour
 Fin
```

Passage d'un tableau en paramètre

Cas d'un tableau à 2 dimensions :

```
Procédure afficherTableau(tab : tableau à 2 dimensions de réels)
    Déclarations
        Variables
            i, j : entier
 Début
    Pour i de 0 à tailleLignes(tab) – 1
        Pour j de 0 à tailleColonnes(tab) – 1
            afficher tab[i, j]
        FinPour
    FinPour
Fin
```

Passage d'un tableau en paramètre

Attention !

Passer un tableau en paramètre revient à transmettre l'adresse de sa première case. Il s'agit donc d'un passage par référence sans avoir à le préciser.

Toute modification d'un tableau faite dans une procédure ou une fonction affecte directement le tableau d'origine : inutile donc de le renvoyer.

Plan

1. Introduction
2. Comprendre ce qu'est un tableau
3. La déclaration d'un tableau
4. L'utilisation d'un tableau
5. Tableau à 2 dimensions
6. La déclaration d'un tableau à 2 dimensions
7. L'utilisation d'un tableau à 2 dimensions
8. Passage d'un tableau en paramètre
9. Chaine de caractères (string)

Chaines de caractère

Une chaîne de caractère est un tableau de caractères, mais pas exactement… car il s'agit d'un type à part : chaine

Déclarations

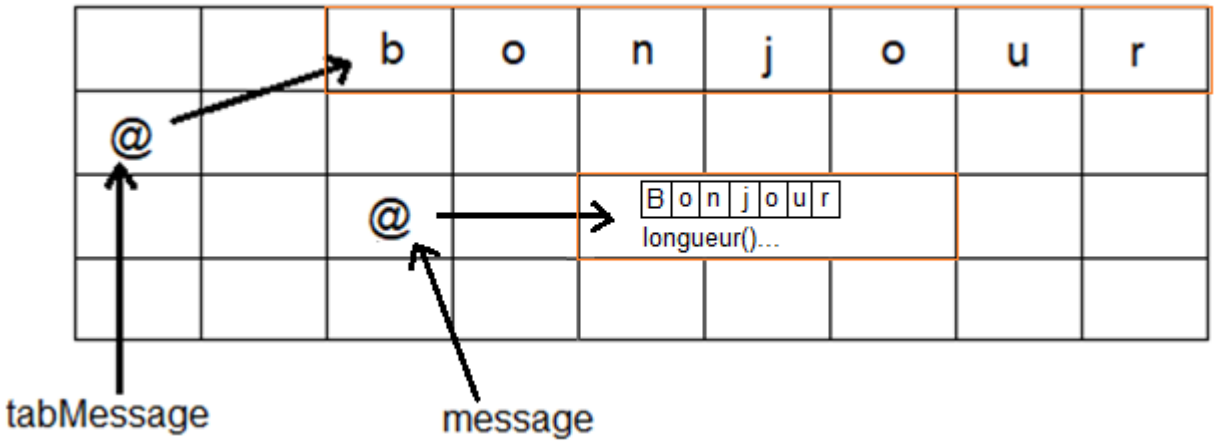
variables

message : chaine

tabMessage : tableau de 6 caractères initialisé à {'B', 'o', 'n', 'j', 'o', 'u', 'r'}

Début

message = "Bonjour"



Chaines de caractère

Ce qui est similaire

- Possibilité de connaître la longueur

```
longueur(message)          //fournira 7
longueur(tabMessage)
```

- Accéder à un caractère par son indice (1er indice à 0)

```
Afficher message[5]        //Affichera 'u'
Afficher tabMessage[5]
```

- Possibilité de parcourir une chaîne dans une itération

```
Pour i de 0 à longueur(tabMessage) - 1
    Afficher message[i]
    Afficher tabMessage[i]
FinPour
```

Chaines de caractère

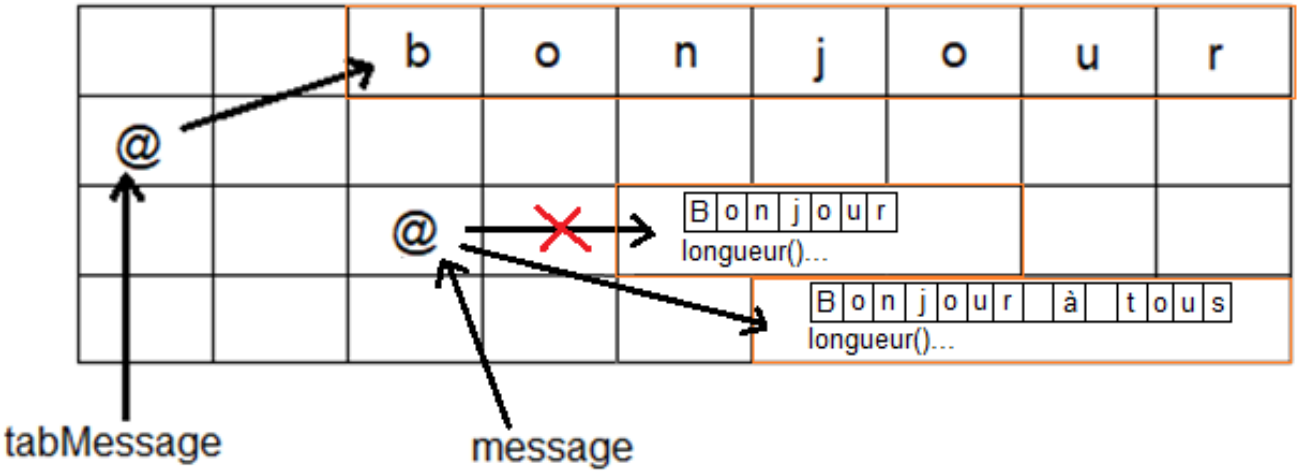
Ce qui est différent

- Une chaîne est immuable

C'est-à-dire qu'on ne peut pas modifier une chaîne !

Si je fais : `message = message + " à tous"`

Il va bien y avoir une concaténation de **"Bonjour"** avec **" à tous"** mais le résultat sera affecté à un nouvel espace mémoire. La variable message pointera désormais vers ce nouvel espace.



Chaines de caractère

Quelques outils permettant de manipuler les chaînes

Je déclare une chaine :

message : chaine

À laquelle j'affecte une valeur;

message = "Bonjour à tous"

- Recherches et inspection

Outil	Résultat
longueur(message)	14
contient(message, "ou")	vrai
indiceDe(message, "ou")	4 (-1 si absent)

Chaines de caractère

Quelques outils permettant de manipuler les chaînes

message = "Bonjour à tous"

- Modification d'une chaine (création d'une nouvelle chaine !)

Outil	Résultat
sousChaine(message, 0, 3) *Indice de départ* *Nombre de caractères à traiter*	"Bon"
enMajuscule(message)	"BONJOUR À TOUS"
enMinuscule(message)	"bonjour à tous"

Remarque : donc si on veut modifier la chaine d'origine il faut faire :

```
message = enMajuscule(message)
```